

GrowthPath Analytics

Initial Technical Framing for MVP Validation

1. Executive Summary

Based on the current concept and prototype, GrowthPath appears technically feasible. The harder question is not whether the platform can be built, but which buyer, channel, and delivery model deserve the first build. The MVP should therefore be designed around evidence gathering: can the trade-area diagnostic create enough clarity and urgency for a local business owner or agency to pay?

The initial build should support a small number of real customer pilots, not a fully scaled SaaS platform. In practical terms, the first version should preserve the core diagnostic methodology, make the output credible and easy to act on, and avoid overbuilding agency white-labeling, deep permissions, advanced analytics, or polished AI chat until demand is proven.

Guiding principle

Build the smallest reliable system that can support 5-10 paid validation customers and produce learning. Do not build the system needed for 10,000 customers until the buyer and channel are clear.

2. Current System as I Understand It

The current prototype appears to separate into three logical layers. Keeping these layers explicit will make the product easier to test, refactor, and scale later.

Layer	Purpose	MVP interpretation
Data collection / rules engine	Accepts business inputs, keywords, location, competitors, and pulls data from external APIs and crawlers.	This should become an asynchronous scan job that can be run repeatedly and logged reliably.
Scoring / methodology engine	Transforms raw data into scores, benchmarks, visibility curves, competitor comparisons, and prioritized findings.	This is the defensible product logic. It should be modular, testable, and explainable.
Delivery / experience layer	Presents the result as report, dashboard, checklist, AI narrative, progress tracking, and rescan history.	This should start lean: show the customer the core aha moment and track whether they act.

3. MVP Product Boundary

The first MVP should not try to be the final platform. It should answer a few validation questions while creating enough value for a real customer to pay or commit.

What the MVP should include

- A simple business onboarding flow: business name, address, category, website, target keywords, and competitor inputs.
- A repeatable scan workflow that can run in the background and produce a stable diagnostic output.
- The core trade-area visibility output: near-me intent, city-modifier intent, Map Pack and Local Finder visibility, and competitor comparison.
- A clear two-pillar score: Search Visibility and Website / Conversion Readiness.
- A prioritized action plan with effort, expected impact, owner-friendly explanation, and completion tracking.
- A simple results portal or customer-facing report that can be reviewed in a sales or onboarding conversation.
- A lightweight AI-generated narrative summary, if it materially improves comprehension.

What should probably wait

- Full agency white-labeling and deep multi-client management.
- Complex role-based access control beyond a simple admin/customer split.
- A polished conversational AI assistant with full chat history and advanced retrieval.
- Mobile applications.
- Advanced billing automation and subscription tier complexity before pricing is validated.
- Sophisticated analytics dashboards that are not needed to create the first buying decision.
- Large-scale infrastructure optimization before scan volume requires it.

4. Proposed Lean Architecture

The architecture should be conventional and boring. The value is in the data methodology and product insight, not in exotic infrastructure.

Component	MVP role	Design note
Frontend	Authenticated portal for entering business details, viewing results, and tracking action items.	Keep UI simple. Prioritize clarity of findings over broad feature coverage.
Backend API	Accounts, businesses, scan requests, results, tasks, and user state.	Minimal REST or RPC API. Avoid premature service fragmentation.
Worker / job runner	Runs scans, API calls, crawls, scoring, report generation, and scheduled rescans.	This is the natural home for the current Python logic.
Database	Stores users, businesses, scans, findings, tasks, competitors, keywords, and rescan history.	A relational schema is appropriate because the product needs auditability and longitudinal comparison.
Object storage	Stores generated reports, raw API response snapshots, crawl artifacts, and exported files.	Useful for debugging, evidence, and customer-facing artifacts.
AI summarization layer	Turns scan output into owner-readable narratives and recommended next steps.	Start with summaries and explanations before building full chat.
Billing	Manual invoice or simple checkout for pilots.	Stripe automation can wait until willingness-to-pay is clearer.

5. Important Design Choices

5.1 Preserve the methodology as a separable engine

The ZCTA / Census-driven trade-area logic, scoring model, and prioritization rules should be isolated from the user interface. That makes the core methodology testable, explainable, and reusable across direct-to-owner and agency channels.

5.2 Treat scans as asynchronous jobs

A full diagnostic touches multiple APIs and may involve latency, retries, failed external calls, and rate limits. The scan should be modeled as a background job with clear statuses: queued, running, partially complete, complete, failed, or needs review.

5.3 Keep raw evidence

The product will be more credible if it can show where conclusions came from. Store enough raw or normalized source data to debug findings, explain scores, and defend the recommendation logic.

5.4 Design for multi-tenancy, but do not overbuild it

It is reasonable to model businesses, users, organizations, and locations cleanly from the start. However, agency features like white-label domains, advanced permissions, client workspaces, and branded report builders should wait until agency demand is validated.

5.5 Make recommendations explainable

The customer should understand why a recommendation appears, what evidence supports it, what effort it requires, and how success will be measured. This matters more than having a long list of findings.

6. MVP Data Model - Conceptual Only

A first-pass model might include the following entities. This is not a schema; it is a vocabulary for discussing product scope.

Entity	What it represents
Organization	The paying customer: owner, agency, or internal pilot account.
Business / Location	The local business being scanned.
Keyword Set	Target services and search terms to evaluate.
Competitor	Local competitors discovered or manually selected.
Scan	One diagnostic run with timestamp, inputs, status, and cost.
Finding	A scored issue or opportunity produced by the diagnostic.
Action Item	A customer-facing task derived from one or more findings.
Metric Snapshot	Rank, GBP, website, traffic, and conversion-related measurements over time.
Narrative Summary	AI-generated or human-reviewed explanation of what changed and what to do next.